HARNESS LAB · MÓDULO 6 · AULA 6.2

Os *3 desfechos*

Entregou, ainda rodando ou falhou: ler o log para achar a camada que faltou. Na falha, a causa está escrita num arquivo, e você corrige a camada, não o código.

`RELATORIO FINAL` `.phases/logs/` `verify-K.log` `Motivo da falha` `desfecho.sh` `/diagnostico-operador depois` `.progress` `--from N`

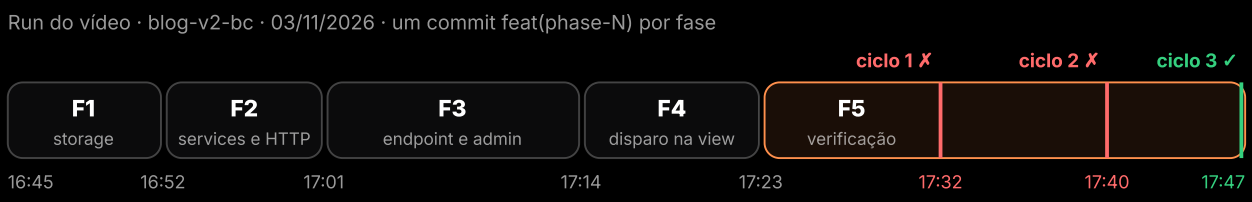
VOCÊ SAI COM

O desfecho do seu run lido (em um comando pelo `loop.sh`, ou à mão pelo ralph da 5.3), o "depois" medido antes de mexer em qualquer coisa e, se o loop falhou, a camada que faltou corrigida e o run disparado de novo a partir da fase que não completou.

Material de apoio · Lucas Souza · Beer and Code

1 A aula em *uma página*

O run da feature-alvo, disparado na 6.1, terminou: **1 hora, 2 minutos e 10 segundos** para as cinco fases do contador de visitas do blog da Beer and Code (`post-view-counter`). Não foi liso. Em alguns momentos houve falha, e quem pegou foram os gates.



As fases 1 a 4 passaram de primeira: gates verdes, commit, próxima. Na última, Verificação transversal, o juiz do gate 3 (a sessão verificadora independente) disse que tinha coisa incompleta e mandou voltar, duas vezes. No terceiro ciclo a suíte ficou verde, a sessão verificadora passou e a fase completou, com o commit `07eb35c`.

Uma hora, e por quê

A feature parecia simples, mas o plano levantou poréns no back-end: estrutura de cache, validação, filtro de bot e uma janela de 24 horas para não contar o mesmo visitante várias vezes. A máquina carregada também pesa no tempo.

Confira no app

Localhost, vários refresh, admin, posts: o card Visitas mostra a contagem por idioma e o total 1. Muitas entradas, uma visita. A regra das 24 horas funcionando na tela, não só no log.

O celular conta a história

No WhatsApp chegou cada passo: feature iniciada, cada fase, os gates verdes, as falhas de cada ciclo e a fase completa. Se tivesse falhado, a mensagem seria "falhou, estou precisando de você".

Human in the loop

Travou, você entra, ajusta e põe para funcionar. Depois o ralph parte da fase que não completou; as que passaram não rodam de novo. Enquanto ele trabalha, você faz outra coisa (no vídeo, passear com o cachorro).

> *"The parts that stay with you are starting it, deciding what "done" means this time, and approving the merge at the end."*
>
> Documentação oficial do Claude Code · Best practices, "Let Claude run without you"

1h 2m

O run do vídeo: 5 fases, de 16:45 a 17:47, a última em 3 ciclos e 23m 51s, numa máquina carregada. É um exemplo, não referência de tempo. A aula fecha pedindo o seu relato nos comentários: o primeiro loop foi de primeira ou teve falha no caminho?

2 Indo *além da aula*

ALÉM DA AULA · TRÊS DESFECHOS, E O SILÊNCIO

DESFECHO	COMO APARECE	OLHE PRIMEIRO
Entregou	`RELATORIO FINAL` sem `Falharam` (`Completadas` e, no re-run, `Puladas`), exit 0	`git log --oneline`: um `feat(phase-N)` por fase que escreveu
Rodando ou esperando	Sem relatório. No limite de uso, "Espera 1/20": dorme até o reset sem gastar ciclo	O processo vive? O log mais novo de `.phases/logs/` cresce?
Falhou	`FALHOU apos 3 ciclos`, `Falharam (1)`, exit 1	O log do gate que reprovou por último

Mais duas assinaturas, sem causa no log: **interrompido** (Ctrl-C ou terminal fechado: o estado diz `failed` com a fase ainda em `running`) e **morto** (SIGKILL, queda, reboot: `running` para sempre). No celular, depende de quem rodou:

DESFECHO	RALPH DA 5.3 (O DO VÍDEO)	PELO LOOP.SH
Entregou	"feature COMPLETA"	"terminou N/T fases"
Falhou	"gate vermelho" a cada ciclo (causa cortada em 200 caracteres), "fase FALHOU", "feature terminou com falhas"	"falhou: F*n* (título), gate *g*, ciclo *k*/3." e a causa
Limite, interrompido, morto	Nada	"esperando limite de uso" (vigia), "interrompido na fase *n*", "morto na fase *n*"

Pelo ralph da 5.3, silêncio pode ser trabalho, limite ou morte; pelo `loop.sh`, a máquina ou o próprio `loop.sh` caiu. Só é "rodando" depois que você vê o processo vivo.

Fonte: `ralph.sh` do bc-harness 0.2.0 (`on_exit`, `wait_for_reset`) e os fixtures; `notify` do `ralph.sh` da 5.3, sem `trap`; `loop.sh` e `vigia.sh` do Módulo 5 (limite e morto lidos no código). O limite semanal, o de Opus e o de Sonnet não são detectados (deduzido do código).

ALÉM DA AULA · RELATÓRIO VERDE E COMMIT SÃO DUAS COISAS

No vídeo, 5 fases e 5 commits `feat(phase-N)`: bate. Fase verde sem commit só é normal se a sessão não escreveu nada no ciclo 1 (`JA IMPLEMENTADA`). Fora disso, desconfie: com um hook de `pre-commit` que recusa (husky, lint-staged), o ralph imprime "Commit criado", sai com 0 e deixa tudo staged. O `desfecho.sh` cruza fases e commits e acusa. E confira no app, como no vídeo: `/verify` faz o Claude subir o app e olhar o resultado, e pega o que teste e typecheck deixam passar.

Fonte: `if ! commit_phase` desliga o `set -e` e ninguém checa o `git commit` (igual nas duas versões do ralph); reproduzido em teste, nunca num run real. `/verify`: doc de skills, "Run and verify your app".

ALÉM DA AULA · PASSOU NO CICLO 3? LEIA POR QUE REPROVOU ANTES

A fase 5 do vídeo tem uma task só, a T10; o juiz numerou 10 no ciclo 1 e 1 no ciclo 2. Compare pela task do PHASES, não pelo número impresso. Ciclo 1: `TASK 10: INCOMPLETE`, o teste de privacidade não cobria o `/feed`: falta real, e o ciclo 2 acrescentou os testes. Ciclo 2: `TASK 1: INCOMPLETE`, "Pint não verificado": o critério pedia "Pint não deixa diff", e o juiz só lê. Critério que só se prova rodando algo o juiz não confirma em ciclo nenhum: lint é o hook da 2.2 ou o comando do gate 2. E ciclo extra pede olhar o diff da fase (6.3): no `lab-agent-seller`, o critério falava de "qualquer nginx à frente" num projeto sem nginx, e no ciclo 3 o agente criou um nginx para passar.

Fonte: `verify-1.log`, `verify-2.log` e `phase-05.md` do vídeo; nginx: `145b2b7`, fatos do Módulo 5.

ALÉM DA AULA · ONDE ESTÁ A CAUSA, E QUAL CAMADA ELA APONTA

O prompt `phase-NN.cycle-K.txt` traz o "Motivo da falha" do ciclo **anterior**. A causa final está só no log do gate que reprovou: `verify-K.log` (G3), `test-K.log` (G2) ou o fim do `cycle-K.log` (G0). Os resumos cortam: a mensagem do vídeo parou em "então o crit"; o `last_error` guarda 70 caracteres.

O QUE O LOG MOSTRA	CAMADA PROVÁVEL	O QUE CORRIGIR
G3 `INCOMPLETE`, mesma task	Prova	Critério que não se prova lendo (4.3)
G2 vermelho no mesmo teste	Prova ou Loop	Teste errado ou instável; fase grande
`permission_denials`	Trava	Barrou o legítimo: a regra. Mexia no teste: o teste
Não escreveu e o juiz reprovou	Regra	Fase mal descrita
G0 sem linha de resultado	Loop	Sessão morta ou teto de tempo (5.2)
Juiz cobriu parte das tasks	Ruído	Rode de novo

Mapa provável, não regra: o diagnóstico da 1.4 com log de verdade. E a correção vai na camada: consertar o código na mão faz o próximo loop errar igual. Teste de aceite errado? O `guarda-aceite.py` da 4.2 barra o Claude, não você: corrija à mão, em commit separado.

Fonte: `ralph.sh` (prompt de correção, `last_error`), conferido nos prompts do vídeo; fatos do Módulo 6 (inferência). Re-rodar resolveu o ruído do juiz em 3 casos, sem garantia.

ALÉM DA AULA · RETOMAR DE ONDE PAROU SEM PAGAR DUAS VEZES

1. **Meça antes.** O `/diagnostico-operador depois` lê o `.phases/logs/`, que o próximo run apaga (só o `.progress` fica). No vídeo, ele leu ali 5 fases, 2 gates vermelhos e 2 ciclos de correção.
2. **Guarde o log.** O `loop.sh` copia para `.loop-logs/<run>/`; com o ralph direto, copie você.
3. **Árvore limpa.** Commite o parcial (o ralph revalida a fase) ou descarte com `git checkout -- .` (arquivo novo, à mão). O `git clean -fd` sugerido apaga o que não está no git nem é ignorado.
4. **Mesmo comando, mesmo PHASES.** As completadas são puladas e a que falhou recomeça no ciclo 1. Editou o PHASES? O carimbo sha256 muda e as fases feitas rodam de novo, pagas. Pelo ralph direto (o da 5.3), `--from N` pula as anteriores a N; o `loop.sh` não repassa opções.

Fonte: `ralph.sh` (as duas versões); README do bc-harness, "Estado e progresso"; git 2.55.

3 Instale e faça *no seu repositório*

Pré-requisito: um run do loop na sua feature-alvo (aula 6.1). O `desfecho.sh` lê o `.phases/state/run.tsv` que o ralph do bc-harness grava (o do `loop.sh` do Módulo 5) e não depende de outra aula. Precisa de bash 4+, git e GNU `date`. No macOS (não testado): `brew install bash coreutils` e o `libexec/gnubin` do coreutils na frente do `PATH`. No Windows, só pelo WSL.

1 Baixe os materiais do curso (uma vez só).

Terminal · na sua pasta pessoal — shell

```
cd ~
git clone https://github.com/beerandcodeteam/harness-lab-materiais
```

Já baixou antes? Atualize:

Terminal · em qualquer pasta — shell

```
git -C ~/harness-lab-materiais pull
```

2 Instale o desfecho.sh e commite.

Cria `.claude/scripts/desfecho.sh`. Commite logo: arquivo novo fora do git suja a árvore, e o pré-voo do próximo run reprova.

Terminal · na raiz do seu projeto — shell

```
~/harness-lab-materiais/instalar.sh 6.2
git add .claude/scripts/desfecho.sh
git commit -m "chore: desfecho.sh da aula 6.2"
```

3 Treine nos desfechos que você ainda não viu, sem gastar token.

Estados de run: um real (entregou), os outros gerados com o ralph de verdade e uma sessão simulada. O `|| echo` mostra o exit, que também diz o desfecho.

Terminal · na raiz do seu projeto — shell

```
A=~/harness-lab-materiais/modulo-6-a-entrega
F=$A/aula-6.2-os-3-desfechos/modelos/fixtures
.claude/scripts/desfecho.sh "$F/run-entregou.tsv"
.claude/scripts/desfecho.sh "$F/run-falhou-g3/" || echo "exit $?"
.claude/scripts/desfecho.sh "$F/run-interrompido.tsv" || echo "exit $?"
.claude/scripts/desfecho.sh "$F/run-morto.tsv" || echo "exit $?"
```

Exit: `0` entregou, `10` rodando, `11` esperando o limite, `20` falhou, `21` interrompido, `22` morto, `2` não achou o `run.tsv`. A saída do `run-falhou-g3`, explicada, está no fim desta seção.

4 Leia o desfecho do seu run.

Pelo `loop.sh` do Módulo 5 (ele chama o ralph do bc-harness, que grava o `run.tsv`), o run ficou em

`.loop-logs/`. Leia o mais novo:

Terminal · na raiz do seu projeto — shell

```
R=$(ls -td .loop-logs/run-*/ | head -1)
.claude/scripts/desfecho.sh "$R" || echo "exit $?"
```

Rodou o ralph do bc-harness direto? Sem argumento, ele lê o `.phases/` do projeto:

Terminal · na raiz do seu projeto — shell

```
.claude/scripts/desfecho.sh || echo "exit $?"
```

Entregou? Confira o cruzamento de fases e commits e o app. Falhou? Anote o gate, o arquivo da causa final e a camada provável.

5

Rodou o ralph do vídeo? Leia à mão.

O `.claude/scripts/ralph.sh` da aula 5.3 (o do vídeo, com aviso no WhatsApp) não grava o `run.tsv`: o `desfecho.sh` responde "não achei", exit 2. Ele deixa o mesmo `.phases/`. Processo vivo? O que ficou pronto? Qual a causa?

Terminal · na raiz do seu projeto — shell

```
pgrep -af "[r]alph.sh" || echo "nenhum ralph vivo"
cat .phases/.progress
ls -t .phases/logs | head -4
grep -h "INCOMPLETE" .phases/logs/*.verify-*.log
```

O `.progress` tem uma fase completada por linha; os logs mais novos são o último ciclo. Gate 2, gate 0, o "Motivo da falha" e a cópia do `.phases/` antes de rodar de novo: `modelos/ler-o-run-a-mao.md`.

6

Meça antes de mexer em qualquer coisa.

O plugin é o da aula 1.1; se já está instalado, os comandos só confirmam.

Terminal — claude

```
claude plugin marketplace add beerandcodeteam/harness-lab-materiais
claude plugin install diagnostico-operador@harness-lab
```

Ou dentro do Claude Code:

Dentro do Claude Code — sessão

```
/plugin marketplace add beerandcodeteam/harness-lab-materiais
/plugin install diagnostico-operador@harness-lab
/reload-plugins
```

A medição grava em `.diagnostico-operador/`. Fora do git e do exclude, ela suja a árvore e um `git clean -fd` apaga o seu "antes". Garanta o exclude:

Terminal · na raiz do seu projeto — shell

```
E=$(git rev-parse --git-path info/exclude)
L=/.diagnostico-operador/
grep -qxF $L $E || echo $L >> $E
L=/diagnostico-operador.md
grep -qxF $L $E || echo $L >> $E
git status --short
```

Dentro do Claude Code · na raiz do projeto — sessão

```
/diagnostico-operador depois
```

Se o `.phases/` já é de outro run, acrescente na mesma linha `--logs .loop-logs/<run>/logs`. Esse "depois" é o que a 6.3 compara com o "antes" da 1.1.

7 Falhou? Corrija a camada, não o código.

Numa sessão normal do Claude Code, peça pela camada que o log apontou. Exemplo para o juiz que reprova a mesma task (Prova):

Dentro do Claude Code — sessão

```
> leia o verify log da fase e diga qual critério não se prova lendo
```

Um pedido completo por camada, com o que conferir na resposta: `modelos/pedidos-por-camada.md`. Ele propõe; você aprova e commita a correção sozinha.

8 Rode de novo, a partir da fase que não completou.

O mesmo disparo da 6.1, mesmo PHASES (troque `post-view-counter` pelo slug da sua feature), num terminal próprio (`tmux new -s loop`). Terminal novo perde o `AVISO_CMD`, e sem ele o aviso não sai pelo seu canal. Pelo `loop.sh`:

Terminal · no tmux, no seu projeto real (gasta token) — loop

```
export AVISO_CMD=.claude/scripts/aviso-env.sh
P=.spec/features/post-view-counter/PHASES.md
.claude/scripts/loop.sh "$P"
```

Pelo ralph direto, como no vídeo. É o da 5.3 (`instalar.sh 5.3`, que a 6.1 já mandou rodar). Ele roda sem os freios do shim da 5.2 (`LOOP_TIMEOUT`, `LOOP_BUDGET_USD`), sem o vigia e sem o aviso do `loop.sh` (manda o WhatsApp dele), e o `.phases/` você copia à mão antes (`modelos/ler-o-run-a-mao.md`):

Terminal · no tmux, no seu projeto real (gasta token) — ralph

```
P=.spec/features/post-view-counter/PHASES.md
.claude/scripts/ralph.sh --engine claude "$P"
```

Mexeu no PHASES? No ralph direto, acrescente `--from N` (a fase que falhou). "Funcionou" só depois do relatório.

A saída do desfecho.sh *no run que falhou*

O fixture `run-falhou-g3`, com caminhos encurtados: qual desfecho, que fase e gate, onde está a causa final, que sinal ela dá e a camada provável.

`.claude/scripts/desfecho.sh "$F/run-falhou-g3/"` — saída real (trecho) · não copie do PDF

```
DESFECHO: FALHOU (0/2 fases concluídas)
  F1   FALHOU      ciclo 3/3   G0 ok · G1 ok · G2 ok · G3 FALHOU   Gerador do texto do .ics
Fase 1 · "Gerador do texto do .ics" · falhou no ciclo 3 de 3
  Gate que reprovou: G3, verificação independente (o juiz)
  Causa final: .../run-falhou-g3/logs/phase-01.verify-3.log
    TASK 1: INCOMPLETE: o critério pede DTEND exclusivo e o código grava o último dia
  Sinal: a mesma task INCOMPLETE em todos os 3 ciclos (task 1).
  Camada provável: Prova. Critério que não se prova ou ambíguo (aula 4.3).
  Atenção: .../prompts/phase-01.cycle-3.txt traz a causa do ciclo 2, não a final.  (exit 20)
```

4 Arquivos que *reforçam a aula*

`.claude/scripts/desfecho.sh`

Os três desfechos e as duas assinaturas sem causa num comando: status, PID vivo ou não, fase, gate, arquivo da causa final, camada provável e o cruzamento de fases e commits. Só lê: não roda o loop nem chama o Claude.

No repositório: modulo-6-a-entrega/aula-6.2-os-3-desfechos/projeto/

`modelos/fixtures/`

Oito estados de run para treinar antes do seu: um entregou real, falhas no gate 3, 2 e 0, interrompido, morto, morto na espera do limite e o verde sem commit do pre-commit.

No repositório: modulo-6-a-entrega/aula-6.2-os-3-desfechos/modelos/

`modelos/pedidos-por-camada.md`

"Corrija a camada, não o código" em forma de pedido: um por camada (Prova, Loop, Trava, Regra), cada um proibindo mexer no código da feature e dizendo o que conferir antes de aceitar.

No repositório: modulo-6-a-entrega/aula-6.2-os-3-desfechos/modelos/

`modelos/ler-o-run-a-mao.md`

O desfecho sem `run.tsv`, como no ralph do vídeo: processo vivo, `.progress`, causa por gate, "Motivo da falha" e a cópia do `.phases/` antes de rodar de novo.

No repositório: modulo-6-a-entrega/aula-6.2-os-3-desfechos/modelos/

5 Erros *comuns*

SINTOMA	CAUSA	CORREÇÃO
`desfecho: não achei .../run.tsv`, exit 2	O seu ralph não grava estado (o da aula 5.3, do vídeo), ou você não está na raiz	Leia à mão (passo 5). Pelo `loop.sh`, passe a pasta `.loop-logs/<run>/`

SINTOMA	CAUSA	CORREÇÃO
Celular quieto há horas	Ralph da 5.3 morto ou dormindo no limite (ele não avisa); pelo `loop.sh`, a máquina caiu	O `desfecho.sh` diz MORTO ou ESPERANDO LIMITE; sem `run.tsv`, o `pgrep` do passo 5
Fase verde, sem o `feat(phase-N)`	Hook de `pre-commit` recusou e o ralph disse "Commit criado"	`git status`: está staged. Rode o hook à mão, corrija, commite
O re-run aborta: árvore suja	O parcial da fase que falhou, o script ou o diagnóstico fora do git	Commite ou descarte o parcial; commite o script; exclude (passo 6)
O segundo run refez todas as fases	Você editou o PHASES: o carimbo mudou e o progresso zerou	Ralph da 5.3 direto com `--from N`, sem os freios do `loop.sh`; pelo `loop.sh`, aceite o custo
O mesmo erro no run seguinte	Você consertou o código, não a camada	Critério, teste, fase ou regra: `modelos/pedidos-por-camada.md`

6 Você sai *com*

- [ ] `.claude/scripts/desfecho.sh` commitado e o desfecho do seu run lido, por ele ou à mão: qual dos três, ou interrompido, ou morto.
- [ ] Entregou: um `feat(phase-N)` por fase que escreveu, conferido no `git log`, e a feature conferida no app.
- [ ] O "depois" medido com `/diagnostico-operador depois` antes de qualquer re-run, com o diagnóstico no exclude.
- [ ] Falhou: gate, arquivo da causa final e camada provável anotados; a camada corrigida num commit seu.
- [ ] O segundo run disparado pelo mesmo comando, pulando as fases completadas.

PROMETIDO NA AULA

- ✓ **Os 3 desfechos, "o que fazer em cada caso"** (título e grade da aula): a tabela da seção 2 (p. 3) e o `desfecho.sh` em `projeto/.claude/scripts/`, passos 2 a 4 (p. 5 e 6).
- ✓ **"Ler o log para achar a camada que faltou"** (grade): o mapa do log à camada (p. 4), a camada provável que o `desfecho.sh` imprime (p. 8) e `modelos/ler-o-run-a-mao.md`.
- ✓ **"Se tivesse falhado, chegaria aqui para mim (...) falhou, estou precisando de você"**: o que cada desfecho mostra no terminal e o que chega no celular, pelo ralph da 5.3 e pelo `loop.sh` (p. 3), e onde está a causa inteira, que a mensagem corta (p. 4).
- ✓ **"A gente consegue partir o Ralph da parte onde a gente parou"**: "Retomar de onde parou" (p. 4) e o passo 8 (p. 7).
- ✓ **"Travou, a gente precisa vir aqui ajustar e colocar isso pra funcionar"**: passo 7 (p. 7) e `modelos/pedidos-por-camada.md`.

7 *Glossário*

Desfecho	Como o run terminou: entregou, ainda rodando (ou esperando o limite) ou falhou. Mais duas assinaturas sem causa: interrompido e morto.
Gates 0 a 3	G0: a sessão terminou direito. G1: a sessão escreveu algo (sinal, não veredito). G2: a suíte, rodada pelo ralph. G3: o juiz, uma sessão verificadora que só lê.
Ciclo de correção	Sessão nova para a mesma fase, com a causa do ciclo anterior no prompt ("Motivo da falha"). Três por fase no padrão.
`.progress`	As fases completadas. Sobrevive entre runs e faz o ralph pular o que já passou.
Carimbo (stamp)	O sha256 do arquivo de fases. Mudou o PHASES, o progresso zera.
`run.tsv`	O estado do run que o ralph do bc-harness grava em `.phases/state/`: status, PID, fases, gates, último erro.
Interrompido · morto	Interrompido: Ctrl-C ou terminal fechado, falhou sem causa. Morto: o estado diz que roda e o processo não existe.
Human in the loop	Você nos pontos que ficam com você: disparar, decidir o que é pronto, destravar quando falha, aprovar o merge.

8 Para *ir além*

- Best practices: "Let Claude run without you", as cinco perguntas antes de deixar um run sozinho e quem assina o resultado.
- Run Claude Code programmatically: o que o SIGTERM faz com o turno (exit 143, sem resultado) e onde aparecem os `permission_denials`.
- Errors: as mensagens de limite (sessão, semanal, Opus, Sonnet) e o que cada uma bloqueia até o reset.
- Effective harnesses for long-running agents: o agente que "would look around, see that progress had been made, and declare the job done". É por isso que o juiz confere task a task.
- BC Harness, README em português: as opções do ralph (`--from`, `--keep-going`, `--max-cycles`) e a seção "Estado e progresso".

Próxima: *Aula 6.3 · Revisando o PR do loop*

O loop para no commit. Entregou ou não, agora você revisa o que ele fez, em PRs pequenos, e compara o antes com o depois que acabou de medir.